

SOFTWARE DEVELOPMENT REPORT

Barnacle Cleaning Robot

Arduino Programming, GUI Interface, and AI Features

Integrated Design Project (IDP)

University of Malaya

Year 3 Semester 2, 2025/2026

1. Introduction

This report presents a comprehensive overview of the software development undertaken for the Barnacle Cleaning Robot, an underwater remotely operated vehicle (ROV) designed to autonomously and manually detect and remove barnacle fouling from submerged structures. The software system is a critical component that integrates embedded microcontroller programming, a real-time graphical user interface, and artificial intelligence-based computer vision into a cohesive operational platform.

The software architecture is distributed across multiple ESP32 microcontrollers connected via RS485 serial communication and I2C buses, a Python-based desktop GUI for operator control and monitoring, and a YOLOv8 deep learning model for real-time barnacle detection. The entire system was developed from scratch to meet the unique requirements of underwater ROV operation, including reliable long-distance wired communication through a tether cable, real-time video streaming from a submerged camera, and robust motor control for both locomotion and cleaning mechanisms.

The development process involved significant engineering challenges, including managing half-duplex RS485 communication timing, implementing a custom binary protocol with CRC8 error checking, handling multi-threaded real-time operations in the GUI, achieving stable JPEG video streaming over high-speed serial, and training a custom object detection model on a specialized barnacle dataset. Each of these challenges required careful design decisions and iterative debugging, which are discussed in detail throughout this report.

2. System Architecture Overview

The Barnacle Cleaning Robot employs a distributed multi-microcontroller architecture consisting of four ESP32 boards and a laptop-side control station. This distributed design was chosen to isolate functionally distinct subsystems, reduce wiring complexity within the underwater enclosure, and allow independent development and testing of each module. The communication hierarchy is as follows:

ESP32_1 (Topside Controller) — Located at the surface alongside the laptop, this board serves as the USB-to-RS485 bridge. It receives command frames from the laptop over USB (UART0) and retransmits them down the tether cable via a MAX485 transceiver on UART2. In the reverse direction, it relays telemetry and debug messages from the onboard systems back up to the laptop. This board is the critical link between the operator and the underwater vehicle.

ESP32_2 (Onboard Stepper Controller) — Submerged within the ROV enclosure, this board receives RS485 drive commands from ESP32_1 and translates them into step-and-direction signals for four TB6600 stepper motor drivers. It also acts as an I2C master, forwarding drum motor commands to ESP32_3. Additionally, it implements a state machine-based RS485 frame parser, a software-based real-time step pulse generator, and a 500-millisecond communication failsafe that halts all motors if contact with the surface is lost.

ESP32_3 (Onboard DC Motor Controller) — This board operates as an I2C slave at address 0x10 and controls the JGB37-555 drum cleaning motor via an IBT-4 H-bridge. It receives a single byte (0 for OFF, 1 for ON) from ESP32_2 and applies the corresponding PWM output to the motor driver. The separation of the DC motor controller onto its own ESP32 was a deliberate design choice to electrically isolate the high-current drum motor circuit from the sensitive stepper driver logic, preventing noise-induced step loss.

ESP32-S3 (Camera Module) — An ESP32-S3 with PSRAM and an OV5640 camera sensor captures JPEG frames at 15 FPS and streams them to the laptop over its native USB CDC port at 921600 baud. Each frame is encapsulated in a custom binary protocol with a 4-byte magic header and a big-endian length field, allowing the receiving GUI to reliably extract individual JPEG images from the continuous byte stream.

Laptop Control Station — The laptop runs a Python-based GUI application that simultaneously manages Xbox controller input, serial communication with ESP32_1, camera stream reception and decoding, AI-based barnacle detection, and a full telemetry dashboard with real-time status indicators.

3. Communication Protocol Design

One of the most technically challenging aspects of the project was designing a reliable communication protocol that could operate over a long tether cable in a noisy underwater environment. The system uses RS485 differential signalling, which is well-suited for long-distance communication in electrically noisy environments because it is resistant to common-mode noise. However, RS485 is inherently half-duplex when using a single twisted pair, which introduced significant complexity into the protocol design.

3.1 Frame Structure

The custom binary protocol uses the following frame structure, defined in the shared `protocol.h` header file:

Frame Format: `[0xAA] [0x55] [CMD] [LEN] [PAYLOAD...] [CRC8]`

Header Bytes (0xAA, 0x55): These two bytes form a distinctive start-of-frame marker. The combination 0xAA (10101010 in binary) followed by 0x55 (01010101 in binary) was chosen because it creates a recognizable alternating bit pattern that is statistically unlikely to appear in random noise or mid-payload data. The receiver uses a state machine that transitions from `WAIT_H1` to `WAIT_H2` to `READ_CMD` only when this exact sequence is detected, providing robust frame synchronization even after data corruption or partial frame loss.

Command Byte (CMD): A single byte identifying the command type. Three commands are defined: `CMD_DRIVE` (0x01) for motor speed control, `CMD_DRUM` (0x02) for the cleaning drum toggle, and `CMD_PING` (0x03) for heartbeat keepalive messages.

Length Byte (LEN): Specifies the number of payload bytes that follow. For `CMD_DRIVE`, the length is 2 (two signed 8-bit speed values); for `CMD_DRUM`, the length is 1 (a single on/off byte). The parser validates that `LEN` does not exceed the 32-byte internal buffer, discarding oversized frames to prevent buffer overflow exploits.

CRC8 Error Detection: The final byte is a CRC8 checksum computed over the `CMD`, `LEN`, and `PAYLOAD` bytes using the polynomial 0x07 with an initial value of 0x00. This is the same CRC-8/ITU polynomial used in many industrial protocols. The CRC computation is implemented identically in both the C++ firmware and the Python GUI to ensure cross-platform compatibility. Frames that fail CRC validation are silently discarded, and the receiver state machine resets to `WAIT_H1` to begin scanning for the next valid frame.

3.2 Half-Duplex RS485 Timing

Managing the half-duplex nature of the RS485 bus was a particularly difficult challenge. The MAX485 transceiver has a single Data Enable / Receive Enable pin (DE/RE) that must be driven

HIGH to transmit and LOW to receive. The critical timing issue is that after the last byte of a frame has been written to the UART transmit buffer, the actual transmission on the wire takes additional time to complete. If DE/RE is switched back to receive mode too early, the final bytes of the frame will be truncated on the wire.

The solution employed in the firmware is a three-step transmit sequence: (1) assert DE/RE HIGH to enable the transmitter, (2) write all frame bytes to Serial2 and call Serial2.flush() to block until the hardware UART FIFO is completely empty, and (3) insert a 50-microsecond delay before de-asserting DE/RE back to LOW. The 50-microsecond guard time accounts for the propagation delay through the MAX485 driver and the tether cable. This value was determined experimentally — shorter delays caused occasional frame corruption at the receiving end, while longer delays reduced the effective throughput of the bus.

3.3 RS485 Receiver State Machine

The onboard ESP32_2 implements the RS485 receiver as a six-state finite state machine (FSM) that processes one byte at a time from the UART receive buffer. The states are WAIT_H1, WAIT_H2, READ_CMD, READ_LEN, READ_PAYLOAD, and READ_CRC. This byte-by-byte approach was chosen over a blocking read-until-complete-frame strategy because the stepper motor pulse generation must not be interrupted — the loop() function must execute the step pulse generator on every iteration to maintain smooth motor motion at up to 2000 steps per second.

The state machine is designed to be self-recovering: any unexpected byte at any state causes a transition back to WAIT_H1. This means that even if a frame is corrupted in transit, the receiver automatically resynchronizes on the next valid frame header without requiring any explicit reset or acknowledgement mechanism. This was a deliberate design decision to prioritize real-time responsiveness over guaranteed delivery — at 50 Hz, a single dropped frame is imperceptible to the operator, but a stalled recovery procedure could cause dangerous loss of control.

4. Arduino Programming — Embedded Firmware

The embedded firmware for the Barnacle Cleaning Robot was developed using the Arduino framework for ESP32, which provides a familiar API for hardware peripheral access while leveraging the powerful dual-core Xtensa LX6 processor and its rich set of peripherals. The firmware is divided into three separate programs, each tailored to its specific microcontroller and responsibilities.

4.1 Topside ESP32_1 — USB-to-RS485 Bridge (Topside_ESP1.ino)

The topside ESP32_1 firmware is architecturally the simplest of the three, but its reliability is paramount because it is the single communication gateway between the operator and the underwater vehicle. The entire program consists of a continuous bidirectional byte-forwarding loop between USB Serial (UART0) and RS485 Serial2 (UART2).

Pin Configuration:

- **UART2 TX (to MAX485 DI):** GPIO17
- **UART2 RX (from MAX485 RO):** GPIO16
- **DE/RE Control:** GPIO22 (HIGH = transmit, LOW = receive)

The bridge operates in idle-receive mode by default. When bytes arrive from the laptop over USB, the firmware transitions to transmit mode by asserting GPIO22 HIGH, writes all available bytes to Serial2, calls flush() to ensure complete transmission, waits 50 microseconds for line settling, and then returns to receive mode. This approach ensures that the bus is occupied for the minimum possible time, maximizing the window for receiving telemetry from the onboard systems.

A key difficulty encountered during development was that the original design used UART0 for both USB communication and RS485, which caused a conflict: the MAX485 transceiver was driving 5V logic levels into the ESP32's 3.3V GPIO pins during receive mode, which eventually destroyed the first ESP32_1 board. The solution was to move RS485 to UART2 (GPIO16/17), keeping UART0 exclusively for USB. This also allowed the developer to use the Serial Monitor for debugging without disconnecting the MAX485 module.

4.2 Onboard ESP32_2 — Stepper Motor Controller (OnboardSteppers_ESP2.ino)

The onboard stepper controller is the most complex of the three firmware programs. It simultaneously performs three real-time tasks: parsing incoming RS485 frames, generating step pulses for four independent stepper motors, and managing I2C communication with ESP32_3 for drum motor control.

4.2.1 Four-Motor Step Pulse Generation

The ROV uses a skid-steer (tank drive) configuration with four stepper motors, each driven by a TB6600 stepper driver. The TB6600 requires a step pulse (PUL) and a direction signal (DIR). Each rising edge on PUL advances the motor one microstep. To achieve smooth motion, the firmware must generate a continuous stream of step pulses at precisely timed intervals.

The step pulse generator is implemented as a software-based timing loop rather than using hardware timers or interrupts. This was a deliberate design choice because using four separate hardware timer interrupts would have introduced complex interrupt priority and resource contention issues on the ESP32, and the Arduino framework's timer abstraction does not guarantee deterministic timing when multiple timers fire near-simultaneously.

Each motor is represented by a `Side` struct containing the pulse and direction pins, the current step interval in microseconds, the direction flag, the timestamp of the last pulse edge, and the current pulse state (HIGH or LOW). The `tickSide()` function is called for each motor on every iteration of `loop()`, and it toggles the pulse pin if enough time has elapsed since the last edge. The step interval is calculated from the commanded speed percentage (0–100%) by mapping it to a steps-per-second value between 0 and `MAX_SPS` (2000), and then converting to a half-period in microseconds.

The maximum step rate of 2000 steps per second corresponds to a minimum step interval of 250 microseconds per edge (500 microseconds per full cycle). This rate was determined by measuring the maximum reliable speed of the TB6600 drivers with the specific stepper motors used in the ROV. At higher rates, the motors exhibited resonance and step loss, particularly under the mechanical load of the underwater environment.

4.2.2 Communication Failsafe

A critical safety feature of the onboard firmware is the 500-millisecond communication failsafe. The `lastRxMs` variable records the timestamp of the most recently received valid frame. On every iteration of `loop()`, the firmware checks whether more than 500 milliseconds have elapsed since the last valid frame. If so, all motor speeds are immediately set to zero, and the drum motor is turned off.

This failsafe protects against several dangerous scenarios: the tether cable being severed or damaged, the topside `ESP32_1` losing power or crashing, the laptop application freezing, or the operator accidentally disconnecting the USB cable. Without this failsafe, the ROV would continue executing its last commanded motion indefinitely, potentially causing damage to the structure being cleaned or to the ROV itself. The 500-millisecond timeout was chosen as a balance between responsiveness (detecting a failure quickly) and robustness (not triggering false alarms during brief communication delays).

4.2.3 I2C Master Communication

ESP32_2 communicates with ESP32_3 over I2C at 100 kHz. The drum motor state is forwarded via a simple one-byte write to the slave address 0x10. To avoid flooding the I2C bus, the `sendDrumIfChanged()` function only transmits when the drum state has actually changed. Additionally, ESP32_2 performs a periodic I2C health check every 1000 milliseconds by sending a zero-length transmission to ESP32_3 and checking the return code. The result (`i2c=OK` or `i2c=FAIL`) is sent back up the tether via RS485 as a status message, allowing the GUI to display the health of the entire board chain in real time.

4.3 Onboard ESP32_3 — DC Motor Controller (OnboardDC_ESP3.ino)

The DC motor controller firmware is intentionally simple and robust. It operates as an I2C slave, listening for single-byte commands from ESP32_2. The `onI2CReceive()` callback is triggered by the I2C hardware interrupt whenever data arrives, and it sets the volatile `drumOn` flag accordingly.

The motor is driven through an IBT-4 H-bridge using the ESP32's LEDC PWM peripheral. The `RPWM` pin receives a 20 kHz PWM signal (the frequency chosen to be above the audible range to avoid whining), while `LPWM` is held permanently LOW because the drum only needs to rotate in one direction for cleaning. The `EN` (enable) pin is also controlled to completely disable the H-bridge when the drum is off, preventing any leakage current from causing unintended motor creep.

The `applyDrum()` function includes a change-detection guard: it only writes to the hardware pins if the requested state differs from the last applied state. This prevents unnecessary GPIO toggling and the associated electrical noise. The function is called from `loop()` at approximately 100 Hz (every 10 milliseconds), which provides a fast response to I2C commands without overwhelming the hardware.

4.4 Camera Module — ESP32-S3 (OnboardCam_S3.ino)

The camera module uses an ESP32-S3 with OPI PSRAM and an OV5640 camera sensor. The ESP32-S3 was chosen specifically because of its native USB CDC (Communication Device Class) support, which allows it to appear as a virtual COM port on the laptop without requiring an external USB-to-UART converter. This is critical because the camera data rate (921600 baud) would overwhelm a standard USB-UART bridge's buffer.

The camera is configured to capture at VGA resolution (640×480) with JPEG compression at quality level 12 (lower values mean higher quality but larger file sizes). Two frame buffers are allocated in PSRAM to enable double-buffering: while one buffer is being transmitted over USB, the camera hardware can fill the other. The `CAMERA_GRAB_LATEST` mode ensures that the most recent frame is always available, discarding older buffered frames to minimize latency.

Each JPEG frame is encapsulated in a custom streaming protocol with a 4-byte magic header (0xA5, 0x5A, 0xF0, 0x0D) followed by a 4-byte big-endian length field and then the raw JPEG data. The magic bytes were chosen to be distinct from any bytes that commonly appear in JPEG data (JPEG markers use 0xFF followed by a non-zero byte). The receiving GUI validates each extracted JPEG by checking that it begins with 0xFF 0xD8 (JPEG SOI marker) and ends with 0xFF 0xD9 (JPEG EOI marker), discarding any corrupted frames.

A significant difficulty encountered with the OV5640 sensor was inconsistent colour rendering — early captures showed a strong green tint and incorrect white balance. The solution was to explicitly configure the sensor's auto white balance, auto exposure, and auto gain control registers during initialization, rather than relying on the driver's default settings. Additionally, the physical mounting orientation of the camera required a 180-degree image rotation, which was implemented in software on the laptop side (using PIL's `rotate()` function) because the OV5640's hardware `vflip` and `hmirror` registers proved unreliable across different ESP-IDF driver versions.

5. Graphical User Interface (GUI)

The Barnacle Cleaning Robot is controlled through a custom-built desktop application developed in Python using the Tkinter framework. The GUI serves as the central command and monitoring station, integrating real-time controller input, serial communication, live video streaming, AI detection overlay, and comprehensive telemetry display into a single cohesive interface. The application is located in `GUI/gui.py` and consists of approximately 1,134 lines of Python code.

5.1 Multi-Threaded Architecture

The GUI employs a carefully designed multi-threaded architecture to ensure that the user interface remains responsive while simultaneously handling time-critical I/O operations. The application runs four concurrent threads:

- **Main Thread (Tkinter event loop):** Owns the Pygame joystick instance, polls the Xbox controller every 30 milliseconds, updates all UI widgets, renders the camera feed, and handles keyboard input. All Tkinter widget updates must occur on this thread because Tkinter is not thread-safe.
- **IoThread (Serial I/O):** A dedicated daemon thread that handles all communication with ESP32_1 over serial. It reads the shared state (motor speeds and drum status) under a threading lock, builds and transmits command frames at 50 Hz, and reads any incoming telemetry/debug messages from the tether.
- **CamThread (Camera Stream):** A daemon thread that reads raw bytes from the camera's serial port at 921600 baud and parses the custom frame protocol to extract individual JPEG images. Successfully decoded frames are stored in the shared Bus object under a lock.
- **DetectorThread (AI Inference):** A daemon thread that loads the YOLOv8 model and performs object detection on camera frames submitted via an internal queue. Detection results (bounding boxes, confidence scores, class names) are written back to the shared Bus object.

Thread synchronization is handled through a shared Bus dataclass that contains all state variables protected by a single `threading.Lock`. This coarse-grained locking strategy was chosen over fine-grained per-variable locks because the critical sections are very short (reading or writing a few primitive values), and the simplicity of a single lock eliminates the risk of deadlocks. The Bus also includes a thread-safe Queue for log messages, allowing any thread to enqueue log entries that the main thread dequeues and displays in the Event Log panel.

5.2 Xbox Controller Integration

The GUI uses the Pygame library to interface with an Xbox controller for intuitive tank-drive operation of the ROV. The controller mapping implements the following scheme:

- **Left Trigger (LT):** Analog axis (0–100%) controlling left-side forward speed
- **Right Trigger (RT):** Analog axis (0–100%) controlling right-side forward speed
- **Left Bumper (LB):** Digital button for left-side reverse at 100%
- **Right Bumper (RB):** Digital button for right-side reverse at 100%
- **A Button:** Toggle drum motor on/off
- **B Button:** Emergency stop (zeroes all motors and disables drum)

The trigger-to-percentage conversion uses a 5% deadzone to eliminate controller drift. The raw Pygame trigger axis value ranges from -1.0 (fully released) to +1.0 (fully depressed) and is normalized to 0–100 using the formula: $pct = ((raw + 1.0) / 2.0) \times 100$. Values below 5% are clamped to zero. The net drive command for each side is computed as the trigger value minus 100 if the corresponding bumper is held, allowing fine-grained bidirectional speed control through a combination of analog and digital inputs.

The drum toggle uses edge detection: the A button state is compared against the previous frame's state, and the drum is toggled only on a rising edge (button newly pressed). This prevents the drum from rapidly toggling while the button is held down. The emergency stop overrides all other inputs — when the B button is pressed, both motor speeds are forced to zero and the drum is disabled, regardless of the trigger and bumper states.

As a fallback, the GUI also supports keyboard input: the Up/Down arrow keys control the left side, W/S keys control the right side, Enter toggles the drum, and Escape triggers the emergency stop. This allows basic operation even without a gamepad connected.

5.3 Dark-Themed Professional UI Design

The GUI uses a custom dark colour palette inspired by modern IDE themes (such as Visual Studio Code's Dark+ theme) to reduce eye strain during extended operation sessions. The colour scheme is defined in a centralized dictionary with semantic colour names: background (#1E1E1E), surface (#262626), card (#2E2E2E), edge (#3A3A3A), text (#E0E0E0), highlight (#17BEBB, a teal accent), OK (#28D17C, green), warning (#E2B93B, amber), error (#FF4D4F, red), and AI accent (#00FFCC, cyan-green).

The application uses the ttk (themed Tkinter) widget set with extensive custom styling via the ttk.Style API. Over 25 custom styles are defined, covering labels, buttons, progress bars, comboboxes, scrollbars, and indicator badges. On Windows, the application additionally calls the undocumented DwmSetWindowAttribute API via ctypes to apply a dark title bar and border colour, achieving a fully native dark-mode appearance that matches the rest of the application.

The layout uses a PanedWindow-based split design with resizable panes. The main window is divided vertically into an upper content area and a lower event log panel. The upper area is further split horizontally into the live camera feed (left) and the telemetry dashboard (right). This layout allows the operator to resize the panels according to their preferences — for example,

expanding the camera feed during inspection and expanding the telemetry panel during diagnostic operations.

A zoom feature (Ctrl+Scroll or Ctrl+=/−/0) dynamically scales all fonts and padding throughout the interface, allowing operators to adjust readability for different monitor sizes and viewing distances. The zoom level is persistently applied to all widgets by recalculating a scale factor and reapplying all ttk styles.

5.4 Real-Time Telemetry Dashboard

The right-hand telemetry panel provides a comprehensive real-time view of the system state, organized into several sections:

- **Controller Status:** Displays the connected controller’s name and a visual indicator showing whether a gamepad is detected.
- **Trigger Bars:** Horizontal progress bars showing the current LT and RT analog values (0–100%), with numeric readouts. LB and RB bumper states are shown as ON/OFF badge indicators.
- **Drive Values:** Large-format signed numeric displays (+000 to +100 or -100) for the left and right motor drive commands, providing immediate visual confirmation of the commanded motion.
- **Drum and E-Stop Indicators:** Colour-coded badge indicators showing drum ON/OFF (green/grey) and emergency stop OK/ESTOP (green/red) states.
- **Board Health Dots:** Three colour-coded dots (ESP32_1, ESP32_2, ESP32_3) that indicate the communication health of each board in the chain. Green indicates recent communication within the last 1.5 seconds, amber indicates stale communication (1.5–4.0 seconds), and red indicates communication loss (>4.0 seconds).
- **AI Detection Panel:** Shows the current AI status (OFF/LOADING/ACTIVE), the number of detected barnacles, and the average detection confidence score.

The telemetry dashboard updates at approximately 33 Hz (every 30 milliseconds), synchronized with the controller polling rate. This update frequency provides a smooth, responsive feel without consuming excessive CPU time.

5.5 Live Camera Feed and Snapshot Feature

The camera feed is displayed in a Tkinter Canvas widget that dynamically scales the incoming JPEG frames to fit the available space while maintaining aspect ratio. Each frame is decoded from the raw JPEG bytes using PIL (Python Imaging Library), rotated 180 degrees to correct for the camera’s physical mounting orientation, scaled to the canvas dimensions using bilinear interpolation, and then converted to a Tkinter-compatible PhotoImage for display.

When AI detection is enabled, detection bounding boxes are overlaid onto the frame before display using PIL's ImageDraw module. Each detection is drawn as a cyan-green rectangle with a label showing the class name and confidence percentage. The `draw_detections()` function uses the Consolas monospace font at 14 points for labels, with a solid background fill behind the text for readability against varied underwater imagery.

A Snapshot button allows the operator to save the current camera frame as a JPEG file to the `snapshots/` directory with a timestamped filename. This feature is useful for documenting the condition of structures before and after cleaning, creating inspection records, and capturing frames for future AI model training.

The camera frame rate is measured using a sliding window: the timestamps of all frames received within the last 1.0 second are stored in a list, and the count of entries in this list is displayed as the FPS indicator in the status bar. This provides a responsive and accurate FPS measurement without the noise of single-frame timing.

5.6 Event Log

The bottom panel contains a scrollable event log implemented as a disabled Tkinter Text widget with a dark theme (black background, light grey text, Consolas monospace font). Log entries are timestamped in HH:MM:SS format and cover all significant events: serial connection/disconnection, controller detection, drum state changes, incoming telemetry messages from ESP32_2 (prefixed with "<-"), AI model loading, detection errors, snapshot saves, and any error conditions.

The log is designed to be append-only and auto-scrolling: new entries are always visible at the bottom. The log widget is kept in a disabled state to prevent accidental editing, and is temporarily enabled only when appending new entries. This design ensures that the log serves as a reliable audit trail of all system events during an operational session.

5.7 Serial Port Auto-Detection

The GUI includes an intelligent serial port auto-detection feature that scans for ESP32-compatible USB-serial adapters by matching against a list of known chipset keywords: CP210x, CH340, CH343, Silicon Labs, USB-Enhanced-SERIAL, USB JTAG, Espressif, and ESP32. It also checks the hardware ID (VID:PID) for the Espressif vendor ID (303A). This eliminates the need for operators to manually identify COM port numbers, which can change between sessions depending on the order in which USB devices are connected.

6. Artificial Intelligence — Barnacle Detection

A key feature of the Barnacle Cleaning Robot is its integrated AI-based barnacle detection system, which uses a deep learning object detection model to identify and localize barnacle colonies in the live camera feed. This capability enables the operator to visualize barnacle coverage in real time, assess cleaning effectiveness, and potentially enable semi-autonomous cleaning operations in future iterations.

6.1 Dataset Preparation

The barnacle detection model was trained on a custom dataset sourced from Roboflow (project: barnacles-lnd34). The dataset contains annotated images of various barnacle species found in marine environments, including Chthamalus, Amphibalanus, Austromegabalanus, Catomerus, Semibalanus, and other common fouling species. Each image is annotated with bounding box labels in YOLO format (class_id, center_x, center_y, width, height, all normalized to 0–1).

The dataset is organized into three splits — training, validation, and test — as defined in the data.yaml configuration file. The dataset defines a single object class ("barnacles"), treating all barnacle species as a unified class. This decision was made because the cleaning robot does not need to distinguish between species; it only needs to identify the presence and location of barnacle fouling. A single-class model also trains more efficiently and achieves higher accuracy than a multi-class model when the inter-class visual differences are subtle, as is the case with closely related barnacle species.

6.2 Model Architecture and Training

The detection model uses YOLOv8s (You Only Look Once, version 8, small variant) from the Ultralytics framework. YOLOv8 was selected for its excellent balance between detection accuracy and inference speed, which is critical for real-time operation. The “s” (small) variant was chosen as a compromise between the lightweight “n” (nano) model and the heavier “m” (medium) model, providing sufficient accuracy for underwater barnacle detection while maintaining inference speeds compatible with the 15 FPS camera feed.

The training script (AI/train.py) configures the following hyperparameters:

- **Input Resolution: 960×960 pixels** — Significantly higher than the default 640×640. This was chosen because barnacles are often small, densely packed objects, and higher input resolution preserves the fine spatial detail needed to distinguish individual barnacles in clusters.
- **Epochs: 100 with Patience 25** — Training runs for up to 100 epochs with early stopping if the validation metric does not improve for 25 consecutive epochs. This prevents overfitting while ensuring the model converges fully.

- **Batch Size: 16** — A moderate batch size that balances GPU memory usage with gradient noise reduction.
- **Learning Rate: 0.01 with Cosine Annealing** — The cosine learning rate schedule smoothly reduces the learning rate from 0.01 toward zero, allowing the model to make large updates early in training and fine-grained adjustments toward the end.

The training process applies aggressive data augmentation specifically tuned for small, dense objects in marine environments:

- Mosaic augmentation (probability 1.0) combines four training images into a single composite image, exposing the model to varied scales and contexts in every batch.
- MixUp augmentation (probability 0.15) blends two images with random transparency, acting as a regularizer.
- Copy-Paste augmentation (probability 0.3) copies annotated barnacle instances from one image and pastes them onto another, artificially increasing the density and variety of training examples.
- HSV colour jittering (hue ± 0.015 , saturation ± 0.7 , value ± 0.4) accounts for the variable lighting conditions and water colour encountered in underwater environments.
- Geometric augmentations including rotation ($\pm 10^\circ$), translation ($\pm 10\%$), scale ($0.5\text{--}1.5\times$), horizontal flip (50%), and vertical flip (10%) to improve robustness to camera angle variations.

6.3 Detection Loss Configuration

The YOLO loss function is configured with custom weights for its three components:

- **Box Loss Weight: 7.5** (default is 7.5) — Governs the accuracy of predicted bounding box coordinates. The default value is retained because barnacle bounding boxes are relatively simple (roughly circular clusters).
- **Classification Loss Weight: 0.5** (default is 0.5) — Since there is only one class, the classification head's role is reduced to distinguishing barnacles from background, and the default weight is appropriate.
- **DFL (Distribution Focal Loss) Weight: 1.5** (default is 1.5) — DFL refines the bounding box regression by modelling it as a probability distribution over discretized offset bins, improving localization accuracy for small objects.

6.4 Real-Time Inference Integration

The trained model is integrated into the GUI through the `DetectorThread` class, which runs YOLOv8 inference in a background thread to avoid blocking the UI. The integration pipeline works as follows:

- When AI mode is enabled by the operator (clicking the "AI ON" button), the `DetectorThread` is instantiated with the path to the best-performing model weights (`AI/runs/barnacle_detect_v2/weights/best.pt`, with automatic fallback to `v1` if `v2` is not available).
- On each tick of the main loop (30 ms), if a new camera frame is available and AI is enabled, a copy of the PIL Image is submitted to the detector thread's input queue. The queue has a maximum size of 1, and old unprocessed frames are discarded to ensure the detector always works on the most recent image.
- The detector thread runs model inference with a confidence threshold of 0.15 (intentionally low to catch faint or partially obscured barnacles), an IoU threshold of 0.45 for Non-Maximum Suppression (NMS), agnostic NMS enabled (treating all detections as the same class for suppression purposes), and a maximum of 1000 detections per frame to handle densely fouled surfaces.
- Detection results (bounding box coordinates, confidence scores, and class names) are written back to the shared `Bus` object under a lock. The main thread reads these detections and overlays them onto the camera feed using PIL drawing functions.

This asynchronous architecture means that AI inference does not reduce the camera feed frame rate. If the model takes longer than 30 milliseconds to process a frame (which occurs on systems without a dedicated GPU), the camera feed continues to display at full speed while AI detections are overlaid with a slight delay. The detection results persist on-screen until the next inference completes, providing a stable and non-flickering overlay.

6.5 Challenges in AI Development

Several significant challenges were encountered during the development and integration of the AI system:

- **Dataset Quality and Size:** The initial dataset from Roboflow contained images of barnacles in ideal conditions (well-lit, above water, high resolution). Underwater images taken by the ROV's camera have significantly lower quality due to turbidity, colour distortion (water absorbs red wavelengths), uneven lighting, and motion blur. This domain gap required extensive augmentation (particularly HSV jittering and brightness variation) to bridge the gap between training and deployment conditions.
- **Small Object Detection:** Barnacles can be very small relative to the image frame, especially when the camera is positioned far from the substrate. The high input resolution (960 pixels) and aggressive mosaic augmentation were critical to maintaining detection performance on small targets. Without these, the model frequently missed individual barnacles and only detected large dense clusters.
- **Inference Latency:** On systems without a CUDA-compatible GPU, YOLOv8s inference on a 960×960 input takes approximately 200–400 milliseconds per frame on CPU. This is too slow for real-time overlay at 15 FPS, but the asynchronous thread architecture means the operator still sees a smooth video feed with detection boxes updating 2–5 times per

second. On systems with a GPU, inference drops to 15–30 milliseconds, enabling true real-time detection.

- **False Positive Management:** The low confidence threshold (0.15) was intentionally set to maximize recall (detecting all barnacles), but this increases the false positive rate. Textures such as rough concrete, corroded metal, and algae patches can be mistakenly identified as barnacles. The GUI displays the average confidence score in the telemetry panel, allowing the operator to assess the reliability of the detections and mentally filter low-confidence results.

7. Standalone Xbox Controller Script (xbox_control.py)

In addition to the full GUI application, a lightweight standalone Python script (Topside_ESP1/xbox_control.py) was developed for direct Xbox controller-to-serial communication without the overhead of the GUI. This script was used extensively during early development and hardware testing, when the GUI was not yet available, and continues to serve as a fallback tool for debugging and field testing.

The standalone script implements the identical control mapping and frame protocol as the GUI but in a command-line format. It prints a single continuously updated status line (L=+050 R=+075 drum=ON estop=False) to the terminal, and echoes any debug messages received from ESP32_2. It also includes a --probe mode that displays the raw axis and button values from the controller, which was invaluable for determining the correct axis/button indices for different Xbox controller hardware revisions.

The script includes automatic serial port detection using the same keyword-matching algorithm as the GUI, allowing it to be launched without specifying a COM port when only one ESP32 is connected.

8. Key Technical Difficulties and Solutions

The software development process encountered numerous technical challenges that required creative engineering solutions. The following sections summarize the most significant difficulties and how they were resolved.

8.1 ESP32 Destruction from RS485 Voltage Mismatch

The original circuit design connected the MAX485 transceiver's RO (Receive Output) line directly to the ESP32's UART0 RX pin (GPIO3). The MAX485 outputs 5V logic levels, but the ESP32's GPIOs are rated for 3.3V maximum. While the ESP32 tolerated this overvoltage initially, prolonged operation caused cumulative damage to the GPIO's internal ESD protection diodes, eventually destroying the pin and then the entire chip. The solution involved two changes: (1) moving RS485 communication to UART2 (GPIO16/17) to free UART0 for USB-only use, and (2) adding a resistive voltage divider on the RO line to attenuate the 5V output to approximately 3.3V.

8.2 Stepper Motor Resonance and Step Loss

At higher speeds, the stepper motors exhibited mechanical resonance — a phenomenon where the motor's natural oscillation frequency coincides with the step frequency, causing violent vibration, loss of torque, and missed steps. This was particularly problematic in the underwater environment where the water resistance created higher mechanical loads. The solution was to empirically determine the maximum reliable step rate (2000 steps/second) and cap all speed commands at this value. Microstepping on the TB6600 drivers (set to 1/8 microstep mode) also helped smooth the motion profile and reduce resonance peaks.

8.3 Camera Green Tint and White Balance Issues

Initial camera captures showed a persistent green colour cast that made the video feed difficult to interpret. This was caused by the OV5640 sensor's default register settings not enabling automatic white balance correction. The fix required explicitly setting the sensor's auto white balance, AWB gain control, auto exposure, AEC DSP, and auto gain control registers during camera initialization. This was not documented in the ESP32 Arduino camera examples and was discovered by reading the OV5640 datasheet and experimenting with register settings.

8.4 Thread Safety in the GUI

A subtle and difficult-to-diagnose bug occurred during early GUI development: the application would occasionally freeze or crash when the serial thread and the main thread simultaneously accessed the shared state variables. The root cause was a race condition where the serial thread

was modifying the left/right motor speed values while the main thread was reading them for display. The solution was the introduction of the Bus dataclass with a single threading.Lock protecting all shared state access. All reads and writes to Bus members are wrapped in "with self.bus.lock:" context managers, ensuring mutual exclusion.

8.5 JPEG Stream Corruption and Recovery

The camera stream occasionally produced corrupted JPEG frames due to USB CDC buffer overruns, particularly when the laptop was under heavy CPU load (e.g., during AI inference). Corrupted frames caused PIL's Image.open() to throw exceptions or produce garbled images. The solution was a multi-layered validation approach: (1) the frame protocol parser validates the magic header and length field, (2) extracted JPEG data is validated against the SOI (0xFFD8) and EOI (0xFFD9) markers, and (3) the Image.open() call is wrapped in a try/except block that silently discards any frame that fails to decode. This ensures the camera feed displays only valid frames, with corrupted frames simply resulting in a held/repeated previous frame rather than a crash or visual artefact.

9. Conclusion

The software system developed for the Barnacle Cleaning Robot represents a comprehensive integration of embedded firmware, desktop application development, and artificial intelligence. The distributed multi-microcontroller architecture provides clean separation of concerns between communication, locomotion, and auxiliary motor control. The custom RS485 protocol with CRC8 error detection ensures reliable command transmission over the tether cable in a challenging underwater environment. The multi-threaded Python GUI delivers a professional, responsive operator experience with real-time video, telemetry, and AI-enhanced barnacle detection. The YOLOv8-based detection model, trained on a custom barnacle dataset with carefully tuned augmentation, provides practical object detection capability that enhances the operator's situational awareness during cleaning operations.

Throughout the development process, numerous technical challenges were encountered and overcome, including hardware-level voltage incompatibilities, real-time stepper motor control constraints, camera sensor calibration issues, multi-threaded synchronization bugs, and the domain gap between training and deployment data for the AI model. Each of these challenges demanded a combination of theoretical understanding, systematic debugging, and practical engineering judgement to resolve.

The resulting software system successfully enables an operator to control the ROV with intuitive gamepad input, monitor all subsystem health in real time, view a live underwater camera feed, and leverage AI to identify barnacle fouling — all from a single desktop application. This work demonstrates the feasibility of combining low-cost ESP32 microcontrollers, open-source AI frameworks, and Python-based tooling to create a capable underwater robotics platform for marine maintenance applications.